AI Image Generator

2. Requirements

Version 2.3
23 October 2026

Project owner: Frazile AI
Platform: NVIDIA GeForce RTX 3090, Windows 11, Python 3.12

Images, video and music on one GPU: Z-Image Turbo, SDXL fine-tunes, Qwen-Image, FLUX.1, LoRAs, LTX-Video, Wan 2.1, MusicGen and Real-ESRGAN, served locally with Gradio in the Frazile AI studio UI, with per-image GPU telemetry recorded in SQLite and an Admin panel that manages models, LoRAs and downloads and shows live system statistics.

2. Requirements

2.1 Functional requirements

ID	Requirement	Status
FR-01	Generate an image from a text prompt using a locally hosted diffusion model on the GPU.	Done
FR-02	Provide a browser interface where a prompt can be typed and the result is displayed.	Done
FR-03	Keep the model loaded between requests so repeat generations take seconds, not minutes.	Done
FR-04	Control aspect ratio, resolution, orientation, quality, fidelity, style, count and seed from words in the prompt, with no separate controls.	Done
FR-05	Show how the prompt was interpreted before generation starts.	Done
FR-06	Produce genuine 4K (3840 px) and 8K (7680 px) output when requested, and exact sizes such as 3840x2160.	Done
FR-07	Display, for every image, the GPU used, time taken, VRAM, temperature, power, clocks and software versions.	Done
FR-08	Persist every generation with its settings and telemetry in a database that can be browsed later.	Done
FR-09	Provide a History view with search, aggregate statistics and click-to-view of past images.	Done
FR-10	Save every generated image, native and upscaled, as a PNG file on disk.	Done
FR-11	Support a negative prompt describing what to avoid.	Done
FR-12	Provide command-line equivalents for generation, upscaling and database queries.	Done
FR-13	Offer several models with different strengths and let the prompt or a dropdown choose between them; switch models without restarting.	Done (v2.0)
FR-14	Choose a suitable model automatically from the style words in the prompt.	Done (v2.0)
FR-15	Accept user-supplied SDXL checkpoints and LoRAs from local folders.	Done (v2.0)
FR-16	Support emphasis weighting such as (word:1.3) and prompts longer than 77 tokens.	Done (v2.0)
FR-17	Upscale an already generated image from the UI without regenerating it, recording the result on the same history row.	Done (v2.0)
FR-18	Present the application in Frazile AI branding with a full-width layout.	Done (v2.0)
FR-19	Provide an Admin panel with live CPU, GPU, memory, storage, network, power and software information.	Done (v2.0)
FR-20	Run the whole application as a GPU-enabled Docker container.	Done, untested on the development machine (no Docker installed)
FR-21	Add models and LoRAs from the Admin panel: Hugging Face repos and files, Civitai links, direct URLs, local files and uploads, with background downloads and progress.	Done (v2.1)
FR-22	Edit any model's defaults and any LoRA's family, trigger words and weight; enable, disable, load, unload, delete and choose the start-up model from the Admin panel.	Done (v2.1)
FR-23	Record which family a LoRA was trained for, route Auto to a matching model and never apply a LoRA to the wrong family.	Done (v2.1)
FR-24	Show the application's own log and generation statistics per model in the Admin panel.	Done (v2.1)
FR-25	Protect destructive Admin actions with a confirmation and an optional password when the server is reachable from other machines.	Done (v2.1)
FR-26	Generate a short video clip from a text prompt, or from a text prompt plus a start image, with a choice of size, length, steps, guidance and seed, saved as an MP4 that browsers play.	Done (v2.2)

ID	Requirement	Status
FR-27	Upscale a generated clip frame by frame to Full HD, 2K or 4K with Real-ESRGAN, and lay a generated music track under it.	Done (v2.2)
FR-28	Generate a music track from a text prompt with a choice of length, guidance, temperature and seed, saved as WAV.	Done (v2.2)
FR-29	Manage video and music models like image models (registry, Admin editing, background download, load, unload, start-up) and record every clip and track in History with telemetry.	Done (v2.2)

2.2 Non-functional requirements

ID	Requirement	How it is met
NFR-01	Runs fully offline after the first model download.	Model weights cached in the Hugging Face cache; upscaler weights stored in models/.
NFR-02	A 1 megapixel image generates in under 15 seconds on an RTX 3090.	Measured 2.0 to 10.5 s depending on model, steps and size.
NFR-03	Peak VRAM stays under 16 GB so 16 GB cards can run every SDXL-family model.	Measured peak 13.2 GB at 832x1216 with prompt weighting; 10.5 GB at 1024x1024 without.
NFR-04	Upscaling must never exceed VRAM regardless of output size.	512 px tiles with 32 px overlap; memory is bounded per tile.
NFR-05	Switching models must free the previous model's VRAM.	ModelManager drops all references, runs gc and empties the CUDA cache; measured 6.58 GB to 0.01 GB resident.
NFR-06	The server binds to localhost only unless deliberately configured otherwise.	Gradio launched on 127.0.0.1:7860; the Docker image binds 0.0.0.0 inside the container only.
NFR-07	All persistent state is in plain files that can be backed up by copying.	PNG files in outputs/, SQLite file generations.db, weights in models/ and the HF cache.
NFR-08	The database schema can evolve without losing data.	db.connect() adds missing columns automatically.
NFR-09	The Admin panel must not slow the GPU work.	A refresh costs about 0.7 s after the first; folder sizes and thread counts are cached for 60 and 30 s.
NFR-10	Code is dependency-light and readable: no framework beyond Gradio and diffusers.	Fourteen Python modules, about 4,300 lines in total.
NFR-11	Downloads must not block generation or the interface.	Downloads run in a two-worker thread pool; the GPU work and the statistics refreshes use separate Gradio queues.
NFR-12	Admin edits must survive restarts and be inspectable.	models/registry.json, written atomically; the built-in list stays in code and only overrides are stored.
NFR-13	Video and music must share the single GPU with the image models without manual unloading.	media.prepare_for(): a video model and an image model never share the card; a music model stays next to an image model while VRAM allows; the image model reloads on the next image.
NFR-14	Video and audio export must not depend on a system ffmpeg install.	imageio-ffmpeg ships a static ffmpeg binary; video_io.py streams frames to it over a pipe so no clip is held in RAM.

2.3 Hardware requirements

Component	Minimum	Recommended	Verified on
GPU	NVIDIA, 12 GB VRAM, compute capability 7.0+ (SDXL family); 24 GB for Z-Image Turbo and FLUX	24 GB	RTX 3090, 24 GB
System RAM	16 GB	32 GB	16 GB
Disk	40 GB free (Z-Image Turbo 20 GB plus its 12 GB bf16 cache, 64 MB upscaler, Python environment, outputs)	120 GB: every SDXL model is 6.5 GB and FLUX is 33 GB	250 GB free, 71 GB used by models

Component	Minimum	Recommended	Verified on
CPU	Any x86-64	6 cores or more	Ryzen 5 3500

Cards below 12 GB can run the command-line script with CPU offload, but the web app assumes an SDXL pipeline fits in VRAM. FLUX on a card under 40 GB loads its transformer in 4-bit and its T5 encoder in 8-bit with per-model CPU offload.

2.4 Software requirements

Package	Version used	Purpose
Python	3.12.0	Runtime
torch	2.14.1+cu126	Tensor library with CUDA 12.6
torchvision	0.29.1+cu126	Required by transformers image processors and spandrel
diffusers	0.41.0	SDXL and FLUX pipelines, schedulers, LoRA loading
transformers	5.20.2	CLIP and T5 text encoders
accelerate	1.15.0	Device placement and CPU offload
safetensors	0.8.0	Safe weight loading
huggingface_hub	1.33.0	Model download and cache
peft	0.21.2	LoRA adapters on the UNet and text encoders
compel	2.5.1	Prompt weighting and long prompts for SDXL
bitsandbytes	0.50.2	4-bit and 8-bit quantisation for FLUX
sentencepiece, protobuf	0.2.2, 7.36.2	T5 tokenizer used by FLUX
gradio	6.30.0	Web interface, theme, timer
spandrel	0.4.2	Loads the Real-ESRGAN architecture and weights
pandas	3.0.6	History table in the UI
psutil	7.2.2	CPU, memory, disk and network statistics for the Admin panel
Pillow	12.3.0	Image I/O, resizing, cropping, figure assembly
numpy	2.5.4	Tile arrays for the upscaler
reportlab	5.0.1	Builds this documentation (development only)
NVIDIA driver	610.88	Also provides nvidia-smi for telemetry

2.5 Model assets

Asset	Source	Size	Location
stabilityai/stable-diffusion-xl-base-1.0 (fp16)	Hugging Face Hub	6.7 GB	HF cache
RunDiffusion/Juggernaut-XL-v9 (fp16)	Hugging Face Hub	6.5 GB	HF cache
SG161222/RealVisXL_V5.0 (fp16)	Hugging Face Hub	6.5 GB	HF cache
Lykon/dreamshaper-xl-v2-turbo (fp16)	Hugging Face Hub	6.5 GB	HF cache
playgroundai/playground-v2.5-1024px-aesthetic (fp16)	Hugging Face Hub	6.5 GB	HF cache
cagliostrolab/animagine-xl-3.1	Hugging Face Hub	6.5 GB	HF cache
Tongyi-MAI/Z-Image-Turbo (bf16)	Hugging Face Hub	20 GB	HF cache (+12 GB bf16 copy in models/cache/)
Qwen/Qwen-Image transformer, VAE, tokenizer	Hugging Face Hub	41 GB	HF cache (+ ~11 GB 4-bit copy in models/cache/)

Asset	Source	Size	Location
diffusers/qwen-image-nf4 text encoder	Hugging Face Hub	5.8 GB	HF cache
lightx2v Qwen-Image-Lightning 8-step V2.0 LoRA	Hugging Face Hub	0.8 GB	HF cache
black-forest-labs/FLUX.1-schnell	Hugging Face Hub (gated, login required)	33 GB	HF cache, not yet downloaded
RealESRGAN_x4plus.pth	github.com/xinntao/Real-ESRGAN releases v0.1.0	64 MB	models/ inside the project
User checkpoints and LoRAs	Civitai or own training	varies	models/checkpoints/, models/loras/

The Hugging Face cache is `%USERPROFILE%\.cache\huggingface\hub` on Windows and the `hf-cache` volume inside Docker.

2.6 Constraints and assumptions

- SDXL-family models are trained at about 1 megapixel. Native generation is therefore capped at a 1536 px long side; larger sizes are reached by upscaling.
- Only one model is resident at a time. Switching takes about 26 s for an SDXL-family model and longer for Z-Image Turbo; the first use of a model downloads it.
- Z-Image Turbo and FLUX.1 schnell have no negative prompt and no guidance scale, and LoRAs and prompt weighting are not applied to them. FLUX also requires a one-time Hugging Face login because the repository is gated.
- The telemetry module depends on nvidia-smi being on the PATH, which the NVIDIA driver installer guarantees on Windows. If it is missing, telemetry fields are stored as NULL and generation still works.
- The application is single-user and single-GPU. Concurrent requests are queued by Gradio and processed one at a time.
- Each model's licence (see Appendix E) permits local and commercial use with its own use restrictions; the operator is responsible for the content they generate.